COMP 484 - HW10: Chrome DevTools Demo

Arrshan Saravanabavanandam

Project 2 Copy - Rocky the Eridian

Overview

For this assignment I made a copy of my Project 2 (Rocky the Eridian) and added Chrome DevTools examples to it. I created a new file called devtools-demo.js that has all the demo functions, and I added a demo panel to the bottom of the page with buttons so you can trigger each example without having to type anything in the console yourself.

I used these two pages as reference:

```
- https://developer.chrome.com/docs/devtools/console/log/

- https://developer.chrome.com/docs/devtools/javascript/
```

---

1. Message Logging

Log Info

I used console.info() to log a startup message and the Friend_info object when the page loads. In DevTools it shows up with a blue (i) icon. I also added a %c style to make the label stand out.

```
console.info(

"%c[Rocky DevTools] INFO - Page initialized",

"color: #4a9eff; font-weight: bold;"

);

console.info("Friend_info at startup:", Friend_info);
```

[Screenshot: Console showing blue info message with Friend_info object]

Log Warning

I used console.warn() to show a warning when Rocky's happiness gets low. It shows up with a yellow triangle in DevTools.

```
console.warn(

"[Rocky DevTools] WARNING - Rocky is unhappy! happiness =",

Friend_info.happiness

);
```

[Screenshot: Console showing yellow warning message]

Log Error

I used console.error() to simulate an error message. It shows up in red with a stack trace.

```
console.error(
"[Rocky DevTools] ERROR - Rocky's food supply critically low!"
);
```

[Screenshot: Console showing red error message]

Log Table

I used console.table() to display Rocky's stats as a table. It makes it way easier to read compared to just logging the object.

```
console.table([
{ stat: "name", value: Friend_info.name },
{ stat: "weight", value: Friend_info.weight + " kg" },
{ stat: "happiness", value: Friend_info.happiness + " notes/min" },
{ stat: "trust", value: Friend_info.trust + " / 100" },
]);
```

[Screenshot: Console showing stats table with stat/value columns]

Log Group

I used console.group() for an outer group and console.groupCollapsed() for a nested one that is collapsed by default. You can click the arrow to expand it.

```
console.group("[Rocky DevTools] GROUP - Rocky status report");
console.log("Happiness:", Friend_info.happiness);
console.groupCollapsed("Speech Lines");
console.log("Treat:", speechLines.treat);
console.groupEnd();
console.groupEnd();
```

[Screenshot: Console showing expandable group with nested sub-group]

Log Custom

I used %c with CSS strings to style a console message like a badge. The two %c markers let you style different parts of the same message separately.

```
console.log(
"%c Rocky the Eridian %c| COMP 484 DevTools Demo",
"background:#3d5c28; color:#e8dfd0; font-size:14px; padding:4px 8px;",
"background:#1a2b42; color:#c97545; font-size:14px; padding:4px 8px;"
```

```
);
```

[Screenshot: Console showing two-tone styled badge message]

View Messages Logged by the Browser

I used console.count() and console.assert() which are similar to how the browser logs its own internal messages. console.assert() only prints if the condition is false, which is what the browser does for constraint violations.

```
console.count("[Rocky] button click count");

console.assert(Friend_info.happiness >= 0, "happiness should not be negative");

console.assert(false, "this always fires - demo of browser-style assert");
```

[Screenshot: Console showing count and assertion messages]

---

2. Cause Errors

Cause 404 Network Error

I triggered a 404 by calling fetch() on a file that doesn't exist. I also added a hidden img tag pointing to the same fake file. Both show up in the Network tab as red failed requests, and the Console shows a browser-generated 404 message.

```
fetch("images/does-not-exist.png")

.then(function(response) {

if (!response.ok)

console.warn("[Rocky DevTools] 404 - status:", response.status);

});
```

[Screenshot: Network tab showing red 404 row]

[Screenshot: Console showing browser 404 error message]

Cause TypeError

I intentionally called .toUpperCase() on Friend_info.weight which is a number, not a string. That causes a TypeError. I wrapped it in a try/catch so the page doesn't crash, but I still log it with console.error so you can see the stack trace.

```
try {

var bad = Friend_info.weight.toUpperCase(); // TypeError on purpose

} catch (e) {

console.error("[Rocky DevTools] TypeError caught:", e.message);

}
```

[Screenshot: Console showing TypeError with message and stack trace]

Cause Violation

I made a loop that blocks the main thread for 200ms on purpose. Chrome automatically prints a [Violation] message in the Console whenever a task takes longer than about 100ms.

```
var start = Date.now();

while (Date.now() - start < 200) { /* blocking on purpose */ }
```

[Screenshot: Console showing [Violation] long task message from browser]

---

3. Filter Messages

I logged messages with specific labels so each filter method is easy to test. Click 'Run All Filter Demos' in the panel and then try the filters below.

```
console.log ("[FILTER-LOG] plain log message");

console.info ("[FILTER-INFO] info message");

console.warn ("[FILTER-WARN] warning message");

console.error("[FILTER-ERROR] error message");

console.log ("[FILTER-REGEX] use regex FILTER-(WARN|ERROR)");

console.log ("[FILTER-SOURCE] from devtools-demo.js");

console.log ("[FILTER-USER] user message for sidebar filter");
```

Filter by log level

Use the Levels dropdown in the Console toolbar to switch between Default, Verbose, Info, Warnings, and Errors.

[Screenshot: Level dropdown open showing filter options]

Filter by text

Type FILTER-WARN in the filter bar at the top of the Console. Only messages containing that string will show.

[Screenshot: Filter bar with 'FILTER-WARN' typed, only warning visible]

Filter by regular expression

Click the /.*/ button to enable regex mode, then type FILTER-(WARN|ERROR). This matches both the warning and error messages at once.

[Screenshot: Regex toggle enabled, pattern FILTER-(WARN|ERROR) in filter bar]

Filter by message source

Open the Console Sidebar (three dots menu > Show console sidebar). Click on the file name to see only messages from that file.

[Screenshot: Sidebar open showing messages grouped by source file]

Filter by user messages

In the same sidebar, click 'user messages' to see only messages logged by your page scripts, not by the browser itself.

[Screenshot: Sidebar with 'user messages' selected]

---

4. Debugging

Reproduce a Bug

The bug I reproduced is from the original Project 2 code. The issue was that updateFriendInfoInHtml() was being called before checkWeightAndHappinessBeforeUpdating(), so if happiness went negative the DOM would briefly show a negative number before getting clamped to 0. I reproduced this in the reproduceBug() function.

```
Friend_info.happiness = 3;

Friend_info.happiness -= 5; // now -2

updateFriendInfoInHtml(); // BUG: writes -2 to DOM

// console.warn shows DOM has "-2"

checkWeightAndHappinessBeforeUpdating(); // clamps to 0 too late
```

[Screenshot: Console warning showing DOM displayed negative happiness value]

Get Familiar with the Sources UI

The Sources panel has three main parts. The left pane is the file navigator where you can find your project files. The center pane shows the source code with line numbers. The right pane has the debugger controls and shows Breakpoints, Scope, Watch, and Call Stack.

[Screenshot: Sources panel open with devtools-demo.js selected and all three panes visible]

Pause the Code with a Breakpoint

To pause the code I opened devtools-demo.js in the Sources tab and clicked the line number next to 'var step = 1;'. A blue dot appeared meaning the breakpoint was set. Then I clicked 'Run Debuggable Action' on the page and execution paused at that line.

[Screenshot: Sources panel paused at breakpoint with blue highlight on the line]

Set a Line-of-Code Breakpoint

A line-of-code breakpoint is set by clicking directly on the line number in the Sources panel. The gutter turns blue to show the breakpoint is active. You can remove it by clicking again.

[Screenshot: Blue breakpoint dot visible in the line number gutter]

Check Variable Values

While paused you can hover over any variable in the source code to see its current value in a tooltip. You can also type variable names directly into the Console while paused.

[Screenshot: Hovering over 'localHappiness' showing its value in a tooltip]

The Scope Pane

The Scope pane on the right side shows all variables in scope at the current pause point. Local shows variables inside the current function (like step, localHappiness, localWeight). Closure shows variables from the outer script like Friend_info. Global shows everything on the window object.

[Screenshot: Scope pane showing Local variables with their current values]

Watch Expressions

In the Watch panel you can add expressions that update live as you step through the code. I added Friend_info.happiness, Friend_info.trust, and step * 10 as examples. Each one updates automatically every time you press F10 to step to the next line.

[Screenshot: Watch panel showing expressions with live values]

The Console (while paused)

The Console still works while the code is paused at a breakpoint. You can type any JavaScript and it runs in the context of the current function. I typed Friend_info and localHappiness * 2 to verify the values mid-execution.

[Screenshot: Console drawer open while paused, showing a typed expression and result]

Apply a Fix

The fix was simple - just swap the order so the guard runs before the DOM update. This way the DOM never sees a negative value because it gets clamped to 0 first.

```
// FIXED - guard runs BEFORE writing to DOM

Friend_info.happiness -= 5;

checkWeightAndHappinessBeforeUpdating(); // clamp first

updateFriendInfoInHtml(); // DOM always sees 0+
```

[Screenshot: Console confirming DOM shows 0, never negative]

All demo functions are in devtools-demo.js. The page has a panel at the bottom with buttons to trigger each example. Open DevTools (F12) before clicking them.